

LAB 5 . REINFORCEMENT LEARNING AND BIOINSPIRED DECISION MAKING

Exercise 1

Complete the previous fragments of code and observe its execution. Print the optimal policy obtained after training and comment on it.

Optimal Policy:

```
↓ ↓ ↓ ↓ ↓
→ → → → ↓
↓ ↓ X → ↓
→ ↓ ↓ ↓ ↓
→ → → → G
```

The optimal policy works correctly being able to traverse the obstacles.

Exercise 2

Now, include a more complex environment where there are more obstacles:

obstacles = [(1, 2), (1, 1), (2, 1), (3, 1), (3, 3), (4, 3)]

Observe the new learnt optimal policy. What's the problem? Why? Change the code to fix the problem using the same number of episodes.

Optimal Policy:

```
→ → → ↓ ↓
← X X ↓ ↓
↓ X → → ↓
← X ↑ X ↓
↓ ↓ ↑ X G
```

I believe that the epsilon value is too small and the rate of exploration (0.1) is not enough for the agent to traverse all the possible paths, and it chooses a safer option, the paths may also be influenced by the negative rewards of hitting obstacles. For example setting an epsilon of 0.8 lets the agent to set reach the goal correctly in the same number of steps.

Exercise 3

Change the previous example so the robot cannot move to a cell where there is an obstacle. The reward in these situations is still -10. Run again the learning process. What has changed in the Q-table? Why?

The rows corresponding to obstacle positions have values 0, the same as the goal position, because the learning loop never goes in these states, they don't learn anything from them, so the value is 0, down below I will show the q_tables of both exercise 1 and 3.

EX 1 Q-table:

```
[[[-3.06451398e+00 -3.05701399e+00 -3.09673938e+00 -3.01964017e+00]
 [-2.51767850e+00 -2.38872249e+00 -2.53279533e+00 -2.04416353e+00]
 [-1.79721323e+00 -4.71432607e-01 -1.90967964e+00 -1.72136853e+00]
 [-1.10986820e+00 -8.56243465e-01 -1.19610437e+00 -1.14664241e+00]
 [-7.72553056e-01 -6.39164969e-01 -7.23300335e-01 -7.72553056e-01]]

[[-2.47895304e+00 -2.47241504e+00 -2.45004056e+00 -2.39620583e+00]
 [-2.05306085e+00 -1.94744165e+00 -1.97384233e+00 -1.26351111e+00]
 [-1.37373270e+00 -3.39686898e+00 -1.38462089e+00 1.67920428e+00]
 [-7.81323289e-01 3.85671408e+00 -8.99993064e-01 -6.37111315e-01]
 [-4.96134899e-01 2.21657364e+00 -6.38338642e-01 -4.87782010e-01]]

[[-2.00589936e+00 -1.92233293e+00 -1.88961541e+00 -1.88411535e+00]
```

```

[-1.57767328e+00 -1.49982060e+00 -1.59229722e+00 -2.71812700e+00]
[-2.81794250e-01 -2.10499886e-01 -2.07100000e-01 9.54586506e-01]
[-3.30951736e-01 7.87666385e-01 -3.38162239e+00 5.94444572e+00]
[-3.28809296e-01 7.95316612e+00 8.80665968e-01 1.99587383e+00]]

[[-1.33029075e+00 -1.34694323e+00 -1.39368631e+00 -1.28684560e+00]
[-9.06083435e-01 -8.67862412e-01 -9.49929941e-01 8.31080988e-03]
[-2.73705086e+00 -1.49156823e-01 -3.35049220e-01 2.76943779e+00]
[-1.99000000e-01 3.46570464e-01 -1.03097036e-01 6.74621869e+00]
[1.07642901e+00 9.99781525e+00 1.45331163e-01 7.84133557e-01]]

[[-9.37359641e-01 -8.64827525e-01 -8.64827525e-01 -7.20086941e-01]
[-6.10947331e-01 -4.90099501e-01 -5.20684987e-01 1.00099884e+00]
[-1.99000000e-01 -1.99000000e-01 -2.15200000e-01 4.54317045e+00]
[-1.99000000e-01 -1.00000000e-01 -7.43401326e-02 8.78423345e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]]]

```

We can see that all cells have values including obstacle positions except the goal state.

EX3 Q-table:

```

[[[ 9.40919141 4.72413107 9.84114052 12.12399135]
[10.70432891 11.52168312 9.36393676 13.53835367]
[13.07843474 12.82042531 11.68945594 14.72722511]
[14.33936431 15.9123858 12.26046029 15.24834495]
[15.25071949 16.94978494 13.7648074 15.41891339]]

[[ 8.85418186 -3.43806302 -0.37203868 0.75271083]
[ 0. 0. 0. 0. ]
[ 0. 0. 0. 0. ]
[13.86041937 16.75712516 15.33319842 16.97112934]
[15.70689519 17.99495452 15.42182184 16.71069453]]

[[ 0.30402402 -4.31107307 -3.65158762 -3.98260737]
[ 0. 0. 0. 0. ]
[10.43554731 6.35293529 9.88786566 16.38138753]
[14.44046079 15.11229483 13.05201403 17.98418878]
[16.93709513 18.99800891 16.73836792 17.92852374]]

[[-3.65503269 -3.90347787 -3.96009068 -4.21164462]
[ 0. 0. 0. 0. ]
[12.25512441 0.17852476 5.48571702 3.23309637]
[ 0. 0. 0. 0. ]
[17.55120755 19.99946877 18.78735763 18.7319758 ]]

[[-3.5777482 -3.58263136 -3.43716218 -3.18365037]
[-3.1417273 -3.14991405 -3.25833927 -0.9086883 ]
[ 5.79093243 -1.60641876 -3.08296268 -1.60150769]
[ 0. 0. 0. 0. ]
[ 0. 0. 0. 0. ]]]

```

All obstacle and goal states are 0.

Exercise 4

Complete the previous code and run it. Define a policy (Q-values) manually that minimizes the need of energy.

```
Manual Policy for Energy Management:
→ → → C ←
↑ ↑ ↑ ↑ ↑
↑ ↑ ↑ ↑ ↑
↑ ↑ ↑ ↑ ↑
↑ ↑ ↑ ↑ ↑
```

A simple policy was implemented so the agent tries to get close to the docking station.

The policy logic just tries to match the number of rows and columns of the actual state with the position of the charger.

Exercise 5

Implement the Q-learning algorithm to allow our robot to learn a policy that minimizes the need of energy. Define the reward function in terms of the variation of the drive of energy and to avoid obstacles. Compare the learnt policy and the manually defined policy from exercise 4.

Well, the policy implemented in previous exercise did not take into account obstacles, this one does, so the optimal policy is going to look a little bit different.

As done in exercise 2, the epsilon is set to 0.8 and the code works correctly.

```
Learned Policy:
→ → → C ←
↑ X X ↑ ↑
↑ X → ↑ ↑
↑ X ↑ X ↑
↑ → ↑ X ↑
```

Exercise 6

Extend the previous code with a new robot's motivation to interact with humans, and it has an associated need or drive called socialization. Also, the robot has a new action interact that satiates the need to socialize, i.e., drive = 0.0, when it is executed next to a person. When the robot is in the same cell as a person, the external stimulus of the new motivation has a value of +10, otherwise it is 0. Define the location of several people:

Exercise 7 (Optional)

Considering exercise 6, modify the code to allow our robot to dynamically learn a policy when the dominant motivation is to recharge its batteries and to interact with people. Notice that the dominant motivation can change from one iteration to another.

Exercise 8

What are the main drawbacks of Q-learning when applied to real robots? Propose a critical analysis of Q-learning that answers this question.

Well it looks like while q learning has advantages like simplicity, and no need for an actual environment, these are also its disadvantages, with real world applications where the states and actions are so complex, q tables will grow too large, and it would be hard and inefficient to compute these values.

Also q learning only takes discrete values in states and the solution is affected immensely by rewards, the epsilon and learning factor, which in small examples may be easier to control, but with larger data the tuning of these parameters can be insane.